

# Heston Stochastic Volatility Model

## Mathematical Implementation and Class Map

HestonVolCalibrator project

May 14, 2026

## Contents

|           |                                                           |           |
|-----------|-----------------------------------------------------------|-----------|
| <b>1</b>  | <b>Preface and scope</b>                                  | <b>2</b>  |
| <b>2</b>  | <b>Notation and conventions</b>                           | <b>2</b>  |
| <b>3</b>  | <b>Heston model dynamics</b>                              | <b>2</b>  |
| <b>4</b>  | <b>Characteristic-function pricing</b>                    | <b>3</b>  |
| 4.1       | Albrecher “little trap” characteristic function . . . . . | 4         |
| 4.2       | Integration . . . . .                                     | 4         |
| <b>5</b>  | <b>Implied volatility inversion</b>                       | <b>4</b>  |
| <b>6</b>  | <b>Calibration objective</b>                              | <b>5</b>  |
| <b>7</b>  | <b>Parameter transforms</b>                               | <b>6</b>  |
| <b>8</b>  | <b>Optimization pipeline</b>                              | <b>6</b>  |
| 8.1       | Nelder–Mead simplex . . . . .                             | 6         |
| 8.2       | Genetic algorithm . . . . .                               | 7         |
| 8.3       | BFGS (unconstrained, transformed space) . . . . .         | 7         |
| 8.4       | L-BFGS-B (box-constrained, raw space) . . . . .           | 8         |
| 8.5       | Comparison of the four optimizers . . . . .               | 8         |
| 8.6       | Pipeline orchestration . . . . .                          | 8         |
| <b>9</b>  | <b>Quote cleaning pipeline</b>                            | <b>9</b>  |
| <b>10</b> | <b>Curve abstraction</b>                                  | <b>10</b> |
| <b>11</b> | <b>Surface construction and service layer</b>             | <b>10</b> |
| <b>12</b> | <b>Web API and frontend (brief)</b>                       | <b>11</b> |
| <b>13</b> | <b>Project layout</b>                                     | <b>11</b> |

# 1 Preface and scope

This note documents the mathematics underlying the `HestonVolCalibrator` codebase and maps each formula to the C# class or file that implements it. In scope: Heston (1993) risk-neutral dynamics, characteristic-function pricing in the Albrecher “little trap” form, Gil-Pelaez probability inversion, Black–Scholes implied volatility, the vega-weighted calibration objective, parameter transforms, and the four optimization back-ends (Nelder–Mead, GA, BFGS, L-BFGS-B). A short section on the quote-cleaning pipeline, curve abstraction, and the web/API surface is included. Out of scope: American exercise, dividends as discrete cash, FFT-based pricing, Monte Carlo path generation, and Greek calculations beyond Black–Scholes vega.

## 2 Notation and conventions

- Time  $t$  in years; ACT/365F implicit at the data-loader level.
- Continuous compounding throughout:  $D(t) = e^{-rt}$ ,  $D_q(t) = e^{-qt}$ .
- Spot  $S_t$ , strike  $K$ , maturity  $T$ , risk-free rate  $r$ , dividend yield  $q$ .
- Variance state  $v_t \geq 0$ ; instantaneous variance, not volatility.
- Heston parameter vector (canonical order used throughout the codebase):

$$\theta = [\kappa, \theta, \sigma, \rho, v_0].$$

We overload the symbol  $\theta$  for both the long-run variance and the parameter vector; context resolves the ambiguity. The codebase resolves it with the field name `Theta`.

- Sign conventions:  $\rho \in (-1, 1)$ ,  $\kappa, \theta, \sigma, v_0 > 0$ .

### Implemented in

- `HestonVolCalibrator/Models/HestonModel.cs` — `HestonModelParams` record.

## 3 Heston model dynamics

Under the risk-neutral measure  $\mathbb{Q}$  with bond numeraire,

$$\begin{aligned} dS_t &= (r - q) S_t dt + \sqrt{v_t} S_t dW_t^{(1)}, \\ dv_t &= \kappa (\theta - v_t) dt + \sigma \sqrt{v_t} dW_t^{(2)}, \\ dW_t^{(1)} dW_t^{(2)} &= \rho dt. \end{aligned}$$

$v_t$  follows a CIR (square-root) diffusion. The Feller condition

$$2\kappa\theta > \sigma^2$$

guarantees  $v_t > 0$  almost surely. The codebase does *not* enforce this constraint: optimizers may freely visit parameters that violate it, since the characteristic-function pricer remains well defined as a complex integral even when  $v$  can touch zero. Box bounds enforce only positivity of  $\kappa, \theta, \sigma, v_0$  and  $|\rho| < 1$  (see §7).

**CIR conditional moments (background).** Given  $v_0$ , the CIR variance has

$$\begin{aligned}\mathbb{E}[v_T | v_0] &= \theta + (v_0 - \theta) e^{-\kappa T}, \\ \text{Var}[v_T | v_0] &= \frac{\sigma^2}{\kappa} \left( v_0 - \theta \left( 1 - \frac{1}{2} e^{-\kappa T} \right) \right) e^{-\kappa T} (1 - e^{-\kappa T}) + \dots\end{aligned}$$

The integrated-variance characteristic function is what enters the option pricer above; we do not simulate paths anywhere in this codebase.

### Parameter intuition.

- $\kappa$ : speed of mean reversion; larger  $\kappa$  flattens the term structure of ATM vols.
- $\theta$ : long-run variance; controls the asymptotic ATM level at long  $T$ .
- $\sigma$ : vol of vol; the dominant driver of smile convexity (kurtosis).
- $\rho$ : leverage / spot-vol correlation; controls smile skew.
- $v_0$ : short-end ATM vol level.

### Implemented in

- `HestonVolCalibrator/Models/HestonModel.cs` — `HestonModelParams` record and the static `HestonPricer`.

## 4 Characteristic-function pricing

European call prices follow from the Heston (1993) decomposition

$$C(S, K, T) = S e^{-qT} P_1 - K e^{-rT} P_2,$$

where the risk-neutral probabilities  $P_j$  are obtained from the characteristic functions  $f_j(\phi)$  via the Gil-Pelaez inversion

$$P_j = \frac{1}{2} + \frac{1}{\pi} \int_0^\infty \text{Re} \left[ \frac{e^{-i\phi \ln K} f_j(\phi)}{i\phi} \right] d\phi = \frac{1}{2} + \frac{1}{\pi} \int_0^\infty \frac{\text{Im} [e^{-i\phi \ln K} f_j(\phi)]}{\phi} d\phi.$$

The second identity uses  $1/(i\phi) = -i/\phi$  so that for any complex  $z$ ,

$$\text{Re} \left[ \frac{z}{i\phi} \right] = \text{Re} \left[ \frac{-iz}{\phi} \right] = \frac{\text{Im}[z]}{\phi}.$$

This is the form actually integrated by the code: the integrand is  $\text{Im}[e^{-i\phi \ln K} f_j(\phi)]/\phi$ , **not**  $\text{Re}[\cdot]$ . Confusing the two flips the sign of  $P_1, P_2$  around  $1/2$  and gives prices that look superficially plausible at the money but diverge from intrinsic at the wings.

Explicitly, with  $f_j(\phi) = \alpha_j(\phi) + i\beta_j(\phi)$  and  $e^{-i\phi \ln K} = \cos(\phi \ln K) - i \sin(\phi \ln K)$ , the integrand simplifies to

$$\frac{\text{Im} [e^{-i\phi \ln K} f_j(\phi)]}{\phi} = \frac{\cos(\phi \ln K) \beta_j(\phi) - \sin(\phi \ln K) \alpha_j(\phi)}{\phi},$$

which matches the inner-loop expression in `ComputeProb`:

```
double cosK = Math.Cos(-phi * logK); // cos(phi*lnK)
double sinK = Math.Sin(-phi * logK); // -sin(phi*lnK)
double imag = cosK * cf.Imaginary + sinK * cf.Real;
sum += imag / phi;
```

## 4.1 Albrecher “little trap” characteristic function

For  $j \in \{1, 2\}$  define  $u_1 = +\frac{1}{2}$ ,  $u_2 = -\frac{1}{2}$ ,  $b_1 = \kappa - \rho\sigma$ ,  $b_2 = \kappa$ . Let  $a = \kappa\theta$  and

$$\begin{aligned} d &= \sqrt{(b_j - \rho\sigma i\phi)^2 + \sigma^2(\phi^2 - 2u_j i\phi)}, \\ g &= \frac{b_j - \rho\sigma i\phi - d}{b_j - \rho\sigma i\phi + d}, \quad (\text{little trap sign}) \\ D(\phi; T) &= \frac{b_j - \rho\sigma i\phi - d}{\sigma^2} \cdot \frac{1 - e^{-dT}}{1 - g e^{-dT}}, \\ C(\phi; T) &= (r - q) i\phi T + \frac{a}{\sigma^2} \left[ (b_j - \rho\sigma i\phi - d) T - 2 \ln \frac{1 - g e^{-dT}}{1 - g} \right], \\ f_j(\phi) &= \exp(C(\phi; T) + D(\phi; T) v_0 + i\phi \ln S). \end{aligned}$$

The Albrecher form is numerically stable because  $g$  has modulus  $\leq 1$  on the integration contour, avoiding the branch crossings of the original Heston (1993) parameterisation. The relevant snippet (note the sign of  $d$  inside  $g$ ):

```
Complex bPart = b - rho * sigma * iPhi;
Complex d = Complex.Sqrt(bPart*bPart + sigma*sigma*(phi*phi - 2.0*u*iPhi));
Complex numG = bPart - d; // numerator: b - rho*sigma*i*phi - d
Complex G = numG / (bPart + d); // denominator: b - rho*sigma*i*phi + d
```

## 4.2 Integration

The integral is computed with the composite midpoint rule on  $\phi \in [0, \phi_{\max}]$  with  $N$  panels:

$$\int_0^{\phi_{\max}} g(\phi) d\phi \approx \Delta\phi \sum_{k=1}^N g((k - \frac{1}{2})\Delta\phi), \quad \Delta\phi = \phi_{\max}/N.$$

Defaults:  $N = 1000$ ,  $\phi_{\max} = 100$ . The midpoint rule avoids the removable singularity of the integrand at  $\phi = 0$ .

Put prices use put-call parity:

$$P(S, K, T) = C(S, K, T) - S e^{-qT} + K e^{-rT}.$$

## Implemented in

- `HestonVolCalibrator/Models/HestonModel.cs` — `HestonPricer.CharFunc` (the  $f_j$  above), `HestonPricer.ComputeProb` (Gil-Pelaez midpoint quadrature), `HestonPricer.CallPrice` and `HestonPricer.PutPrice`.

## 5 Implied volatility inversion

For a call price  $C$ , the Black–Scholes implied volatility  $\sigma_{BS}$  solves

$$C = S e^{-qT} \Phi(d_1) - K e^{-rT} \Phi(d_2), \quad d_{1,2} = \frac{\ln(S e^{-qT} / (K e^{-rT})) \pm \frac{1}{2} \sigma^2 T}{\sigma \sqrt{T}}.$$

The map  $\sigma \mapsto C(\sigma)$  is strictly increasing on  $(0, \infty)$  with  $\partial C / \partial \sigma = \nu > 0$ , so Newton’s method converges from any positive starting point provided  $C$  is strictly inside the no-arbitrage bounds. The implementation performs at most 100 iterations and exits early on either a small parameter

step or a vanishing vega. The codebase implements  $\Phi$  via the Abramowitz–Stegun rational approximation of erf, and the vega

$$\nu = S e^{-qT} \varphi(d_1) \sqrt{T}, \quad \varphi(x) = \frac{1}{\sqrt{2\pi}} e^{-x^2/2}.$$

Newton iterates are safeguarded by:

- rejecting prices at or below intrinsic ( $C \leq \max(Se^{-qT} - Ke^{-rT}, 0) + 10^{-9}$ );
- a Brenner–Subrahmanyam ATM seed  $\sigma_0 = \sqrt{2\pi/T} C / (S e^{-qT})$ , clipped to  $[0.001, 5]$ ;
- per-step clipping of  $\sigma$  to the same interval;
- early exit when  $|\sigma_{n+1} - \sigma_n| < 10^{-9}$  or when  $\nu < 10^{-12}$ .

The data-loader’s `ImpliedVolSolver` converts puts to equivalent call prices through put-call parity, rejects strikes outside  $[0.7S, 1.3S]$ , and discards implied vols outside  $[0.01, 2.0]$ .

### Implemented in

- `HestonVolCalibrator/Implementations/BlackScholes.cs` — `NormalCdf`, `NormalPdf`, BS call/put.
- `HestonVolCalibrator/Models/HestonModel.cs` — `HestonPricer.BsImpliedVol` (safeguarded Newton).
- `HestonVolCalibrator.DataLoader/ImpliedVolSolver.cs` — put-to-call parity wrapper and money/IV sanity filters.

## 6 Calibration objective

For each sample point  $i = 1, \dots, N$  on the  $(T_i, K_i)$  grid, let  $\sigma_i^{\text{mkt}}$  be the market implied vol read from the surface and  $\sigma_i^{\text{mod}}(\theta) = \text{HestonPricer.ImpliedVol}(\theta, S, K_i, T_i, r, q)$  be the model’s implied vol at parameters  $\theta$ . The objective is a *vega-weighted* root-mean-square IV error:

$$\mathcal{L}(\theta) = \sqrt{\frac{\sum_{i=1}^N w_i (\sigma_i^{\text{mod}}(\theta) - \sigma_i^{\text{mkt}})^2}{\sum_{i=1}^N w_i}}, \quad w_i = \max(\varphi(d_1^{(i)}) \sqrt{T_i}, 10^{-6}),$$

where  $d_1^{(i)}$  is the Black–Scholes  $d_1$  evaluated at the *market* vol  $\sigma_i^{\text{mkt}}$  (so weights are fixed per call). Note that the implementation uses  $w_i = \varphi(d_1) \sqrt{T}$  (Black–Scholes *normalized* vega up to the  $Se^{-qT}$  factor) rather than full dollar vega; this keeps the weights of comparable magnitude across maturities. Pathological evaluations (NaN/Inf model vol, all weights tiny) return `double.MaxValue` so optimizers can detect them.

**Why vega-weight?** Two interpretations:

1. Heuristic: low-vega quotes (deep OTM, very short or long  $T$ ) carry little information about the local smile shape because small price perturbations cause large IV swings. Weighting by  $\nu$  down-weights them.
2. Statistical: if quote noise is roughly homoskedastic in *price* space, then by the implicit function theorem the IV-space noise has standard deviation  $\propto 1/\nu$ . Weighting by  $\nu$  (squared inside the sum, with  $\nu^2$  absorbing one factor in the residual and one in  $w_i$ ) is the heteroskedasticity correction that recovers the price-space normal-likelihood MLE up to a constant.

The code uses  $w_i = \varphi(d_1)\sqrt{T_i}$  rather than full dollar vega  $Se^{-qT}\varphi(d_1)\sqrt{T}$ ; the  $Se^{-qT}$  factor is the same for every quote at a given  $T$  (after normalization across maturities its omission is immaterial) and keeps the weights  $O(1)$ .

### Implemented in

- `HestonVolCalibrator/Calibration/VegaWeightedObjective.cs` — the objective.
- `HestonVolCalibrator/Calibration/HestonCalibrator.cs` — builds the sample grid from the surface and orchestrates global  $\rightarrow$  gradient calibration.

## 7 Parameter transforms

Unconstrained optimizers (Nelder–Mead, BFGS) operate in a transformed space that automatically enforces the natural domain:

$$\begin{aligned} q_j &= \ln \theta_j & \text{for } j \in \{\kappa, \theta, \sigma, v_0\}, \quad \theta_j > 0, \\ q_\rho &= \operatorname{atanh}(\rho) = \frac{1}{2} \ln \frac{1+\rho}{1-\rho} & \rho \in (-1, 1). \end{aligned}$$

Inverse map:

$$\theta_j = e^{q_j} \quad (j \in \{\kappa, \theta, \sigma, v_0\}), \quad \rho = \tanh(q_\rho).$$

The codebase clamps  $\rho$  to  $[-0.999, 0.999]$  before `atanh` to avoid  $\pm\infty$ . By construction the transformed search space is  $\mathbb{R}^5$ , so the optimizer need not enforce bounds explicitly. L-BFGS-B (§8.4) is the one exception: it operates in raw space so that user-specified box bounds are honoured literally.

**Trade-offs.** Working in transformed space removes constraints but also reshapes the loss landscape:  $\log \kappa$  and  $\log \theta$  make the objective flatter near zero, which can slow gradient methods when the optimum is small. For L-BFGS-B (which has native bound support), raw space is the right choice. The `atanh` transform is essential for  $\rho$ : without it, an unconstrained optimizer routinely overshoots  $\rho \rightarrow \pm 1$  during line search, at which point the characteristic-function denominator  $(1 - \rho^2)$ -like terms blow up.

### Implemented in

- `HestonVolCalibrator/Calibration/ParamTransforms.cs` — Encode/Decode and TransformedObject.

## 8 Optimization pipeline

### 8.1 Nelder–Mead simplex

A custom Nelder–Mead implementation in transformed space. Standard coefficients:

$$\alpha = 1 \text{ (reflection)}, \quad \gamma = 2 \text{ (expansion)}, \quad \rho_c = 0.5 \text{ (contraction)}, \quad \sigma = 0.5 \text{ (shrink)}.$$

The initial simplex perturbs each coordinate by `InitialPerturbation = [0.5, 0.5, 0.5, 0.3, 0.5]` in transformed space. Convergence is by the spread  $f_n - f_0$  of simplex function values against `opts.Tolerance` ( $10^{-8}$  by default). Multi-start: from `Restarts` starting points, each obtained by adding uniform jitter in  $[-1, 1] \cdot \text{RestartJitter}$  with `RestartJitter = [0.8, 0.8, 0.8, 0.5, 0.8]` to the encoded  $q_0$ . The best-so-far is reported through the iteration callback.

## Implemented in

- `HestonVolCalibrator/Calibration/NelderMeadOptimizer.cs` — `RunSimplex` + multi-start driver.

## 8.2 Genetic algorithm

Wraps the `GeneticSharp` library. Real-valued chromosome of length 5, one gene per Heston parameter, each encoded as a 64-bit `FloatingPointChromosome`. The 64-bit width is required: `GeneticSharp`'s `BinaryStringRepresentation` encodes via `BitConverter.GetBytes((long)...)`  which is always 64 bits; using fewer bits with a range that crosses zero (notably  $\rho \in (-1, 1)$ ) yields a chromosome that cannot represent negative values without sign-extension artifacts. Defaults:

- Population: 50 (fixed min = max).
- Selection: elite.
- Crossover: uniform with mixing probability 0.5, overall probability 0.75.
- Mutation: flip-bit, probability 0.25.
- Termination: fixed generation count (default 100, overridable via `GenerationsOverride`).

The objective is minimized by maximizing  $-f$  (`GeneticSharp` maximizes fitness). The GA runs in *raw* parameter space honouring `obj.Lower` and `obj.Upper`.

## Implemented in

- `HestonVolCalibrator/Calibration/GeneticOptimizer.cs` — chromosome construction and GA loop.

## 8.3 BFGS (unconstrained, transformed space)

`Math.NET`'s `BfgsMinimizer` on the transformed objective. Hessian inverse  $H_k$  is updated via the standard BFGS secant

$$H_{k+1} = \left( I - \frac{s_k y_k^\top}{y_k^\top s_k} \right) H_k \left( I - \frac{y_k s_k^\top}{y_k^\top s_k} \right) + \frac{s_k s_k^\top}{y_k^\top s_k}, \quad s_k = q_{k+1} - q_k, \quad y_k = g_{k+1} - g_k.$$

The search direction is  $p_k = -H_k g_k$  and the next iterate is  $q_{k+1} = q_k + \alpha_k p_k$  with  $\alpha_k$  chosen by a Wolfe-condition line search (`Math.NET` internal). Because the objective is evaluated in transformed space,  $g_k = \partial \mathcal{L} / \partial q$  relates to the raw-space gradient via the chain rule with diagonal Jacobian  $\text{diag}(e^{q_1}, e^{q_2}, e^{q_3}, 1 - \tanh^2(q_4), e^{q_5})$ ; the finite-difference recipe handles this implicitly by differentiating the composed function.

Gradients are computed by central differences in transformed space with  $\varepsilon = 10^{-5}$ :

```
qa[i] = orig + eps; double fp = t.Evaluate(qa);
qa[i] = orig - eps; double fm = t.Evaluate(qa);
g[i] = (fp - fm) / (2.0 * eps);
```

Stopping tolerances: gradient  $10^{-6}$ , parameter step  $10^{-8}$ , function progress  $10^{-10}$ .

## Implemented in

- `HestonVolCalibrator/Calibration/BfgsOptimizer.cs` — gradient and minimizer wiring.

## 8.4 L-BFGS-B (box-constrained, raw space)

Math.NET’s `BfgsBMinimizer` on the *raw* objective. The whole point of L-BFGS-B is to respect bounds literally, so we do not compose with `ParamTransforms`. The starting point is clamped strictly inside the box, and the finite-difference gradient uses a *reflective* stencil at the edges:

$$g_i = \frac{f(x + h_+ e_i) - f(x - h_- e_i)}{h_+ + h_-}, \quad \text{with } h_+ = \min(\varepsilon, U_i - x_i), \quad h_- = \min(\varepsilon, x_i - L_i).$$

This keeps both stencil arms inside  $[L_i, U_i]$ . Tolerances and  $\varepsilon = 10^{-5}$  match the unconstrained BFGS.

### Implemented in

- `HestonVolCalibrator/Calibration/BfgsBOptimizer.cs`.

## 8.5 Comparison of the four optimizers

- **Nelder–Mead**: derivative-free, robust to non-smooth wrinkles (e.g. switching between integration branches), cheap per iteration but slow near the optimum (linear convergence at best). Multi-start mitigates the local-minimum problem. Best as a global stage.
- **Genetic**: handles strongly multi-modal landscapes (rare in Heston but possible with sparse data), no gradient, embarrassingly parallel in principle. The library’s deterministic seed cannot be propagated without a custom `IRandomization`; for reproducibility the harness records the convergence trace.
- **BFGS**: super-linear local convergence when the gradient is well-defined. Transformed space avoids constraint handling at the cost of a curved metric. Vulnerable to large finite-difference noise from the integrand sum (mitigated by central differences and the smooth  $f_j$ ).
- **L-BFGS-B**: same convergence as BFGS but in raw space, the right tool when the calibrator must honour business-defined parameter caps (e.g. risk-management bounds on  $v_0$  or  $\sigma$ ). The price is a slightly more delicate finite-difference stencil at the box edges.

The intended use is global  $\rightarrow$  gradient chaining (`Chain = true`): use Nelder–Mead or GA to find a good basin, then refine with BFGS / L-BFGS-B.

## 8.6 Pipeline orchestration

`HestonCalibrator.Calibrate(CalibrationRequest, IVolatilitySurface, onProgress)` sequences the stages:

1. Build the dense  $(T, K)$  sample grid from `surface.Expiries  $\times$  surface.Strikes`.
2. Construct a `VegaWeightedObjective`.
3. Choose the global optimizer per `GlobalMethod` (`None/NelderMead/Genetic`).
4. Choose the gradient optimizer per `GradientMethod` (`None/Bfgs/BfgsB`).
5. If `Chain` is true (default), the gradient stage starts from the global stage’s best; otherwise from the initial guess.
6. Report convergence points through the `onProgress` callback (used by the SSE streaming endpoint).



7. After convergence, evaluate market vs Heston implied-vol matrices on the same grid for diagnostics.

Default bounds come from `CalibrationRequest`:  $\kappa \in [0.01, 20]$ ,  $\theta \in [10^{-4}, 1]$ ,  $\sigma \in [0.01, 5]$ ,  $\rho \in [-0.99, 0.99]$ ,  $v_0 \in [10^{-4}, 1]$ .

### Implemented in

- `HestonVolCalibrator/Calibration/HestonCalibrator.cs` — pipeline driver.
- `HestonVolCalibrator/Calibration/CalibrationContracts.cs` — request/result records and enums.

## 9 Quote cleaning pipeline

`QuoteCleaner.Clean` runs four sequential stages on the raw option chain.

**Stage 1 – bid/ask sanity.** A quote is dropped if any of:

$$\text{bid} \leq 0, \text{ask} \leq 0, \text{ask} < \text{bid}, \text{bid} < \text{MinBid}, \text{mid} \leq 0, \frac{\text{ask} - \text{bid}}{\text{mid}} > \text{MaxRelSpread}.$$

Defaults: `MinBid = 0.05`, `MaxRelSpread = 0.30`.

**Stage 2 – no-arbitrage price bounds (per side).** Using  $D(T), D_q(T)$  from curves:

$$\begin{aligned} \text{Call: } \max(S D_q - K D, 0) &\leq \text{mid} \leq S D_q, \\ \text{Put: } \max(K D - S D_q, 0) &\leq \text{mid} \leq K D. \end{aligned}$$

A small  $\varepsilon = 0.01$  slack is allowed on each side.

**Stage 3 – butterfly convexity in  $K$  (per side, per maturity).** Sort by strike; for each interior triple  $(K_-, K, K_+)$  the central second difference must satisfy

$$C(K_+) - 2C(K) + C(K_-) \geq -\text{tol}, \quad \text{tol} = 0.05 \cdot \max_i C(K_i).$$

Violating mid quotes are dropped. The sweep is repeated up to 5 times to allow chains of arbitrage violations to clear, and the neighbour of a removed point is skipped within a pass to prevent cascading false positives.

**Stage 4 – MAD-based smile outlier rejection (per maturity).** For each maturity  $T$  with at least 5 surviving quotes:

1. Compute IV  $\sigma_i = \text{ImpliedVolSolver.Solve}$  for each surviving quote.
2. Fit an OLS quadratic in log-moneyness  $x_i = \ln(K_i/F_T)$  where  $F_T = S D_q(T)/D(T)$ :

$$\sigma_i \approx a + b x_i + c x_i^2.$$

3. Compute residuals  $r_i = \sigma_i - (a + b x_i + c x_i^2)$ , then

$$\text{MAD} = 1.4826 \cdot \text{median}_i |r_i - \text{median}(r)|.$$

4. Drop any  $i$  with  $|r_i - \text{median}(r)| > k \cdot \text{MAD}$ , default  $k = \text{MadKSigma} = 4.0$ .

The 1.4826 factor makes MAD a consistent estimator of  $\sigma$  under Gaussian residuals.

### Implemented in

- `HestonVolCalibrator.DataLoader/QuoteCleaner.cs` — all four stages and the helpers `TryFitQuadratic` (OLS via Cramer’s rule) and `Median`.

## 10 Curve abstraction

The interfaces and implementations isolate discount, dividend, and forward curves from the rest of the codebase.

- `IDiscountCurve`:  $D(t) = e^{-z(t)t}$ ,  $z(t)$ , and instantaneous forward  $f(t) = -d \ln D / dt$ .
- `IYieldCurve` extends `IDiscountCurve` with  $q(t)$ .
- `IForwardCurve`:  $F(0, t) = S D_q(t) / D(t)$  and log-moneyness  $\ln(K / F(0, t))$ .

`FlatDiscountCurve/FlatYieldCurve` are constant-rate placeholders. `PiecewiseLinearZeroCurve` linearly interpolates the zero rate  $z(t)$  between strictly increasing pillar tenors and extrapolates flat outside. Its instantaneous forward uses

$$f(t) = z(t) + t \frac{dz}{dt},$$

approximated by a centered finite difference on  $z$ .

**Wiring note.** As of this writing the curve layer is consumed only by `QuoteCleaner` (no-arbitrage bounds and forward computation for MAD filtering). Pricing and calibration still take scalar `rate` and `dividendYield` parameters; lifting them to curves is a clean next step but is not yet done.

### Implemented in

- `HestonVolCalibrator/Interfaces/IDiscountCurve.cs` — interfaces and conventions.
- `HestonVolCalibrator/Implementations/Curves.cs` — flat curves, piecewise-linear zero curve, `ForwardCurve`, `CurveFactory`.

## 11 Surface construction and service layer

`SurfaceBuilder.Build` converts an enumeration of `OptionQuote` into a `GridVolatilitySurface` of IVs. Quotes are first grouped by rounded maturity (4 decimals); within each maturity, for each strike the more liquid leg is kept (highest `OpenInterest + Volume`). Implied vol inversion runs through `ImpliedVolSolver`, after which valid  $(T, K, \sigma, \text{IsCall})$  tuples are inserted into the grid.

The web layer wraps this in a `SurfaceService` that caches surfaces by ticker so repeated calibration requests skip Yahoo fetches. `SyntheticSurface` produces a deterministic Heston-generated surface for sanity tests and demos.

### Implemented in

- `HestonVolCalibrator.DataLoader/SurfaceBuilder.cs` — builder, CSV export, console pretty-printer.
- `HestonVolCalibrator.Web/SurfaceService.cs` — HTTP-layer surface cache.
- `HestonVolCalibrator.Web/SyntheticSurface.cs` — synthetic Heston surface generator.

## 12 Web API and frontend (brief)

Minimal-API ASP.NET Core endpoints:

- `POST /api/surface` — fetch quotes, clean, build the IV surface, return the  $(T, K, IV)$  grid and the matching call/put price grids.
- `POST /api/calibrate` — run a full calibration synchronously and return the fitted parameters, RMSE, and IV grids.
- `POST /api/calibrate/stream` — same as above but streams convergence points as Server-Sent Events.
- `POST /api/heston-surface` — evaluate the Heston pricer for a user-supplied parameter set across a  $(T, K)$  grid (no calibration).
- `GET /api/health` — liveness probe.

The frontend is a single-page Plotly app under `wwwroot/` that renders the market and Heston IV surfaces side by side and animates the convergence trace in real time.

### Implemented in

- `HestonVolCalibrator.Web/Program.cs` — endpoint registration and SSE pumping.
- `HestonVolCalibrator.Web/Contracts.cs`, `JsonConverters.cs` — request/response DTOs.
- `HestonVolCalibrator.Web/wwwroot/index.html` and `app.js` — Plotly UI.

## 13 Project layout

- `HestonVolCalibrator/` — model, pricer, calibration, curves. Pure library, no IO.
- `HestonVolCalibrator.DataLoader/` — Yahoo options loader, implied-vol solver, quote cleaner, surface builder.
- `HestonVolCalibrator.Runner/` — CLI entry point that wires `DataLoader` → `Calibrator` → console output.
- `HestonVolCalibrator.Web/` — ASP.NET Core minimal-API service plus Plotly SPA.
- `HestonModel/` — example/sandbox program.

### End-to-end data flow

1. `YahooOptionsLoader` fetches an option chain for a ticker and produces a list of `OptionQuote` records (bid, ask, last, OI, volume, maturity, strike, IsCall).
2. `QuoteCleaner.Clean` filters quotes through the four stages of §9, returning the survivors and a per-reason drop-count diagnostic.
3. `SurfaceBuilder.Build` inverts each surviving quote to IV (via `ImpliedVolSolver`) and assembles a `GridVolatilitySurface`, choosing the most liquid leg at each  $(T, K)$  cell.
4. `HestonCalibrator.Calibrate` consumes the surface and a `CalibrationRequest`, runs the global → gradient pipeline, and returns market and model IV grids alongside the fitted parameters and convergence trace.
5. The web layer plots both grids and the trace in real time via SSE.

## Numerical-stability checks worth running

- Increase  $N$  or  $\phi_{\max}$  in the pricer and verify convergence of  $P_1, P_2$  to  $<10^{-6}$  digit-stability for moderate maturities.
- Round-trip test: generate synthetic data from a known Heston parameter set, calibrate, and verify recovery within  $10^{-3}$  on each parameter.
- Probe the Feller boundary: hand-set  $\sigma$  near  $\sqrt{2\kappa\theta}$  and check the pricer remains finite.
- Test the integrand sign (§4): if at-the-money the model-IV closely matches but deep-OTM diverges, the integrand is likely on the Re branch instead of Im.